Итоговое домашнее задание

Название дисциплины: Специализированные инструменты прикладного программирования

Информация о задании

Форма проверки: задание с проверкой преподавателем.

Цели задания: проверить уровень освоения содержания дисциплины «Специализированные инструменты прикладного программирования» и закрепить навыки написания кода на Java.

Правила приёма работы:
Прикрепите ссылку на выполненное задание в GitHub-репозитории или его аналоге. Код должен быть написан на языке Java с применением Stream API, декомпозирован на функции, методы и классы, логически структурирован.

Задание считается не выполненным, если:

- ссылка на репозиторий с кодом не прикреплена, не содержит ответа на задание или по ссылке отсутствует доступ**;**
- код выдаёт ошибку или даёт неправильный ответ.

Описание задания

Обратите внимание

Задание разделено на три уровня сложности: базовый и два дополнительных. Дополнительные уровни выполняются по желанию.

Допустимы следующие варианты выполнения задания:

- только базовый уровень;
- базовый уровень + уровень 2;
- базовый уровень + уровень 3.

Одновременное выполнение уровней 2 и 3 не требуется, но возможно.

Технические требования

- Java 17+— основной язык программирования.
- Stream API — для обработки коллекций данных.
- Коллекции Java Framework: List или собственная реализация коллекции (для дополнительного уровня сложности).
- Обработка исключений с использованием try-catch и try-with-resources.
- Для хранения исходных данных допускается использование текстовых файлов формата TXT или CSV.
- Проект должен быть структурирован с использованием классов и методов согласно принципам объектно ориентированного программирования.
- GitHub или аналогичный сервис контроля версий — для хранения и предоставления доступа к коду.

Уровень 1. Базовый (выполнение обязательно)

Разработайте консольное Java-приложение для обработки коллекции объектов.

1. Выберите предметную область и создайте класс минимум с тремя полями.
2. Реализуйте консольное меню.
3. Обеспечьте работу программы в цикле до выбора пользователем пункта «Выход»;
4. Реализуйте возможность добавления объектов:
   - вручную через консоль;
   - из файла.
5. Выполните валидацию вводимых данных.
6. Обработайте исключения, возникающие при вводе данных и чтении файла.
7. При работе с файлами используйте конструкцию try-with-resources.
8. Храните объекты в коллекции List.
9. Используйте Stream API для обработки данных:
   - отфильтруйте объекты по одному условию;
   - выполните хотя бы одно преобразование данных через map();
   - отсортируйте данные по одному полю;
   - выведите результат в консоль.
10. Разместите проект в GitHub.

Уровень 2*. Собственная коллекция

***Повышенная сложность (выполнение по желанию)

Дополнительно к базовому уровню реализуйте собственную коллекцию.

На выбор:

- аналог ArrayList;
- аналог LinkedList.

Коллекция должна поддерживать операции:

- add;
- get;
- remove;

- size.

Для аналога **ArrayList** необходимо реализовать автоматическое расширение внутреннего массива.
Для аналога **LinkedList** необходимо реализовать собственный узел (Node) и хранение элементов через ссылки.

Используйте собственную коллекцию в программе вместо стандартной реализации Java и продемонстрируйте:

- добавление элементов;
- получение элемента;
- удаление элемента;
- вывод содержимого коллекции.

Уровень 3. Собственные алгоритмы сортировки и полиморфизм

Высокая сложность (выполнение по желанию)

Дополнительно к базовому уровню:

1. Реализуйте два собственных алгоритма сортировки;
2. Предоставьте пользователю возможность выбора алгоритма сортировки;
3. Используйте обобщения (Generics) в реализации алгоритмов;
4. Использовать полиморфизм, чтобы:
    - создать общий интерфейс SortAlgorithm;
    - реализовать несколько алгоритмов сортировки;
    - выбирать необходимую реализацию во время выполнения программы.
5. Предоставьте пользователю возможность выбора направления сортировки:
    - по возрастанию;
    - по убыванию.

Примеры предметных областей

В качестве предметной области вы можете использовать один из предложенных ниже вариантов:

1. Автомобиль. Поля:
    - мощность;
    - модель;
    - год выпуска.
2. Студент. Поля:
    - имя;
    - средний балл;
    - номер зачетной книжки.
3. Пользователь. Поля:
    - имя;

- email;
- идентификатор пользователя.

4. Автобус. Поля:
   - номер;
   - модель;
   - пробег.
5. Собственная предметная область. Допускается использование собственной предметной области при условии, что класс содержит не менее трех полей.

Пример реализации консольного меню

Использование данного шаблона не является обязательным и приведено исключительно в качестве примера.

```
Scanner scanner = new Scanner(System.in);

while (true) {
   System.out.println("""

           ===== МЕНЮ =====
            1. Добавить данные вручную
            2. Загрузить данные из файла
            3. Обработать данные
            0. Выход
            """);

    System.out.print("Выберите пункт меню: ");

    String input = scanner.nextLine();

    int command;

    try {
       command = Integer.parseInt(input);
    } catch (NumberFormatException e) {
       System.out.println("Ошибка: необходимо ввести число.");
       continue;
    }

    switch (command) {
       case 1 -> {
           System.out.println("Добавление данных вручную");
           // TODO
        }

       case 2 -> {
           System.out.println("Загрузка данных из файла");
           // TODO
        }

       case 3 -> {
           System.out.println("Обработка данных");
           // TODO
```

```
        }

        case 0 -> {
            System.out.println("Завершение работы программы...");
            return;
        }

        default -> System.out.println("Неизвестная команда.");
    }
}
```

Критерии оценки

Максимальное количество баллов за итоговое задание — **60.**

Уровень 1. Базовый (выполнение обязательно) — максимум 40 баллов.

1. Предметная модель и архитектура приложения (максимум 8 баллов):

- Создан класс предметной области с минимум тремя полями **—** 2 балла.
- Код разделён на классы и методы, соблюдена логическая структура приложения **—** 3 балла.
- Соблюдены базовые требования Java Code Style **—** 3 балла.

2. Консольное меню и взаимодействие с пользователем (максимум 8 баллов)

- Реализовано консольное меню **—** 3 балла.
- Приложение работает в цикле до выбора пункта Выход — 3 балла.
- Пользователь может выбирать доступные действия через меню — 2 балла.

3. Работа с файлами и обработка исключений (максимум 8 баллов)

- Реализована загрузка данных из файла — 3 балла.
- Используется конструкция try-with-resources — 2 балла.
- Реализована обработка исключений при чтении файла и вводе данных — 3 балла.

4. Валидация данных (максимум 4 балла): реализована проверка корректности входных данных.

5. Работа с коллекциями (максимум 4 балла)

- Объекты хранятся в коллекции List — 2 балла.
- Реализовано добавление и обработка элементов коллекции — 2 балла.

6. Использование Stream API (максимум 8 баллов)

- Реализована фильтрация данных через Stream API — 2 балла.
- Реализовано преобразование данных через map() — 2 балла.
- Реализована сортировка данных — 2 балла.
- Реализован вывод результатов обработки — 2 балла.

Уровень 2. Повышенная сложность (выполнение по желанию) — максимум 20 баллов. При этом итоговая оценка за задание (сумма оценок за выполнение заданий уровней 1, 2 и 3) не может превышать 60 баллов.

7. Реализация собственной коллекции (максимум 20 баллов)

- Реализованы основные операции (add, get, remove, size) — 8 баллов.

- Корректно реализована внутренняя структура коллекции (динамический массив или связный список) — 6 баллов.
- Собственная коллекция используется в проекте вместо стандартной реализации Java — 4 балла.
- Код читаемый и корректно декомпозирован — 2 балла.

Уровень 3. Высокая сложность (выполнение по желанию) — максимум 20 баллов. При этом итоговая оценка за задание (сумма оценок за выполнение заданий уровней 1, 2 и 3) не может превышать 60 баллов.

8. Реализация алгоритмов сортировки (максимум 20 баллов)

- Реализованы два собственных алгоритма сортировки — 8 баллов.
- Пользователь может выбирать алгоритм сортировки — 2 балла.
- Использованы Generics в реализации сортировки — 4 балла.
- Использован полиморфизм через интерфейс SortAlgorithm — 4 балла.
- Реализован выбор направления сортировки (по возрастанию / убыванию) — 2 балла.